Red Team Analysis — Home SOC Suite (Professional Edition)

This document provides a comprehensive red team analysis of the Home SOC Suite. It evaluates realistic adversarial attack scenarios, identifies detection gaps, and defines architectural improvements required to harden the system. The goal is to support secure design decisions prior to full Python implementation.

1. Slow Baseline Poisoning

Attackers gradually introduce behavior that appears benign until it becomes normalized. Without baseline lifecycle and drift detection, malicious activity becomes accepted as normal. Design Requirement: Implement baseline versioning, confidence scoring, and drift detection mechanisms.

2. Correlation Window Bypass

Attack steps are distributed across long time intervals to avoid triggering rule-based detection. Design Requirement: Support multi-timescale correlation (minutes, hours, days) and persistent state tracking.

3. Trusted Process Abuse

Attackers execute malicious activity through trusted binaries such as PowerShell or rundll32. Design Requirement: Analyze parent-child relationships, command-line arguments, and behavioral deviations.

4. Noise Flooding

High-volume benign activity is generated to obscure malicious signals. Design Requirement: Implement rate anomaly detection and visibility degradation alerts.

5. Collector Suppression

Attackers disable or disrupt telemetry sources to create blind spots. Design Requirement: Monitor collector health using heartbeat signals and expected event rates.

6. Low-and-Slow Command & Control

C2 communication occurs infrequently to avoid detection thresholds. Design Requirement: Track rarity and accumulate risk scores over time rather than relying on thresholds.

7. Archive Tampering

Archived logs are modified post-incident to hide evidence. Design Requirement: Use encrypted archives with signed manifests, hash verification, and chaining.

8. Log Injection Attacks

Malformed or manipulated logs break parsing or mislead detection systems. Design Requirement: Use strict schema validation and track parser confidence levels.

9. Event Reordering and Replay

Events are delayed or replayed to disrupt correlation logic. Design Requirement: Separate observed timestamps from ingestion timestamps and support deduplication.

10. Cross-Collector Inconsistency

Events appear in one data source but not others, indicating possible tampering. Design Requirement: Implement cross-validation rules between collectors.

11. Identity Context Abuse

Attackers leverage different user contexts to hide intent. Design Requirement: Track user, session type, and privilege context as core attributes.

Conclusion

Final Assessment: Without advanced safeguards, the system is vulnerable to timing attacks, trust abuse, and data manipulation. With the recommended architectural improvements, the system evolves into a resilient behavioral detection platform. Key Principle: Security is not determined by visibility alone, but by the system's ability to reason about time, trust, and consistency.